Isaac Summers SRN: 21043693

Digital Controller Design and Performance Evaluation for a Short-Period Aircraft Model

1 Introduction

This report covers the design, implementation and analysis of a digital controller for the short period dynamics of a DC-8 aircraft model. The objective is to meet specific performance criteria through design and simulation in both continuous and discrete domains. Closed loop stability and response characteristics are analysed and tools such as root locus analysis, performance indices, and comparative simulations in MATLAB and python are employed to develop and validate the control strategy.

2 System Discretisation and Sampling Time Analysis

In this section the continuous time model is discretised by considering minimum and relaxed sampling times. In order to implement a digital controller for the aircraft model, the continuous time transfer function described below must be converted into discrete-time.

$$G_p(s) = \frac{0.04s + 4.774}{s^2 + 5.396s + 12.85}$$

The Nyquist sampling theorem requires the sampling at twice the highest frequency in the analogue signal to avoid aliasing, however, in practical control design all signal content does not need to be reconstructed, and more emphasis should be on maintaining the system dynamics within the operational frequency range where the system actively responds to inputs and disturbances. To accurately represent the analogue signal in digital format a widely accepted guideline in control system design is used to choose the sampling frequency f_s (or sampling period T_s), shown in equation 2.2, in which ω_{BW} is the closed loop bandwidth, f_s and T_s can then be calculated using equations 2.3 and 2.4 respectively.

$$\omega_s > 10 \; to \; 25\omega_{BW}$$

Equation 2.1

$$f_s = \omega_s/(2\pi) \qquad T_s = 1/f_s$$

Equation 2.2 *Equation 2.3*

The bandwidth defines the frequency range over which the system operates satisfactorily and actively responds to inputs and disturbances. [1] From MATLAB the bandwidth for the closed loop system is obtained as 4.55 rad/s, from equation 2.2 a multiplier of 25 was chosen as this corresponds to a lower sampling period, therefore, capturing the system dynamics more accurately. This led to a minimum practical sampling frequency of around 18.1 Hz, this value is also rounded to find a relaxed sampling frequency of 20 Hz. This is done because sampling with round numbers generally gives better insight into the system's behaviour and the higher sampling frequency allows for better capture of system dynamics, however, this increase demands a higher computational cost.

Figure 2.1 shows the bode plot with the continuous system and discrete time system with minimum and relaxed sampling frequencies using $25\omega_{BW}$, and a comparison using $10\omega_{BW}$. It can be determined from the bode plot that the gain and phase margin of the initial system are both infinite so error between these metrics can't be analysed. From the bode plot it can be seen that the system using sampling frequency of $10\omega_{BW}$ starts to deviate from continuous at around 7 rad/s, whereas using $25\omega_{BW}$ it starts to deviate at around 25 rad/s, from this it is

clear that $10\omega_{BW}$ isn't viable as it only accurately captures the system dynamics for less than $2\omega_{BW}$, the system with this sampling frequency also deviates before the point of minimum phase which is important to capture for gain stability.

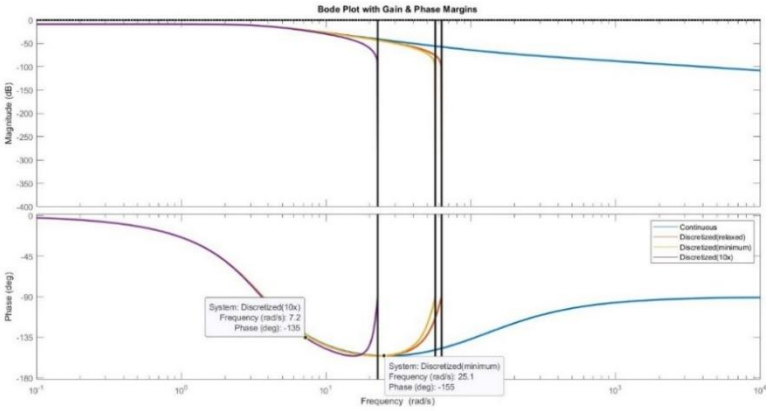


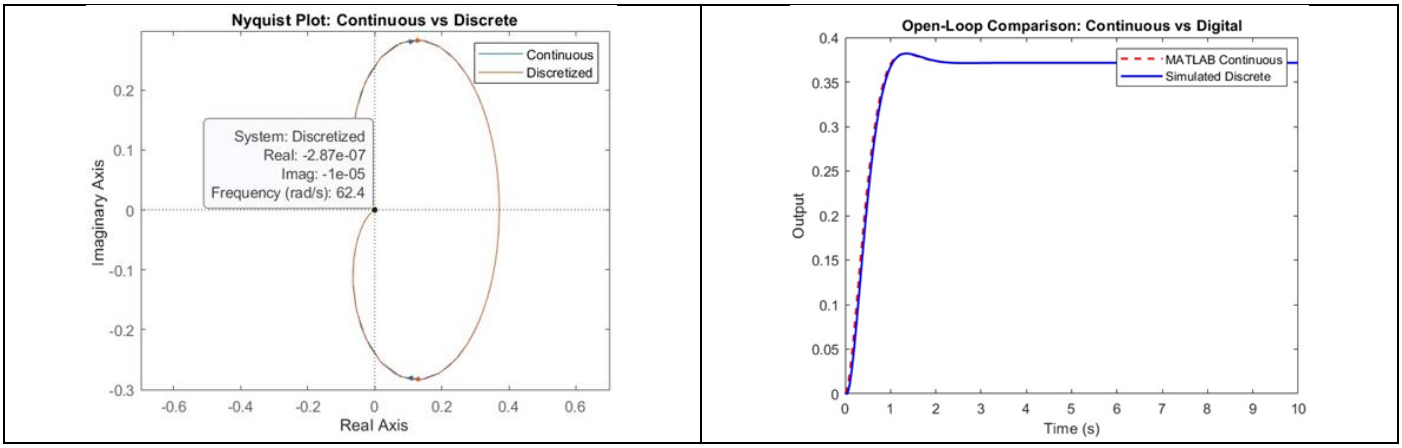
Figure 2.1 – Bode plot with continuous and discrete time systems

Both the relaxed and minimum discretisation are similar. however the relaxed captures system dynamics slightly better whilst having a negligible increase in computational cost. The higher frequencies are not accurately captured by the discretised systems, however, in practical implementation these frequencies likely can't be responded to due to mechanical delays and other inefficiencies, this is also represented by the systems low bandwidth. Sensors also tend to pick up on noise in high frequencies of which the system should ignore and not act on.

Figure 2.2 (a) shows the Nyquist plot for both the continuous and discrete time systems. The discretised system shows very good agreement with the continuous system even following it through the highest frequency of 62.4 rad/s.

The step response of both the continuous and discretised system are shown in figure 2.2 (b), from this it can be seen that the discrete time system follows the continuous very well, initially there is a very slight lag for the discrete system however this delay is negligible and within less than 1% difference.

Overall, this analysis shows that the relaxed sampling time corresponding to T_s = 0.05 seconds (20 Hz) is a practical sampling time which can accurately model the continuous system in the discrete time domain with minimal computational cost.



(a) (b)

Figure 2.2 – Nyquist (a) and step response (b) plot for continuous and discrete time systems

3 Controller Design and Closed-Loop Performance Evaluation

In this section a stable controller is designed in both continuous and discrete domains to meet specified performance metrics as seen in table 3.1. The influence of sampling times on controller effectiveness, damping characteristics, and system stability are examined leading to a justified selection of the optimal controller.

Criteria	Desired value
Settling time t_s (s)	$t_s < 1.8$ (s)
Rise time t_r (s)	$t_r < 2.5$ (s)

Maximum overshoot M_p (%)	$M_p < 5$ (%)
Steady-state error e (%)	e < 2 (%)
Integral of the Square of the Error ISE	Minimum
Integral of the Absolute Error ITAE	Minimum
Integral of the Absolute Control Effort IACE	Minimum
Integral of the Absolute Control Effort Rate IACRE	Minimum

Table 3.1 – System Design Criteria

3.1 Controller Implementation and Setup

The controller in this section is designed assuming that the actuator is ideal and has a transfer function of G_{ac} = 1.0. The discretised system is modelled in MALAB by the linear difference equation Y(z), as implemented in [2], the equation is given as:

$$\frac{Y(z)}{U(z)} = \frac{b_0 z^2 + b_1 z + b_2}{a_0 z^2 + a_1 z + a_2}$$

Where the coefficients b_0, b_1, b_2, a_0, a_1, a_2 are based on the discretised system transfer function, the MATLAB code has been made to automatically take the coefficients from the numerator and denominator of the transfer function and input them into the following equations to both eliminate rounding errors by inputting values directly and make the analysis of the controller at different sampling frequencies easier. The difference equation can then be rewritten in the following form:

$$y[n] = \frac{1}{a_0}(-a_1 y[n-1] - a_2 y[n-2] + b_0 u[n] + b_1 u[n-1] + b_2 u[n-2])$$

This equation represents the systems behaviour in digital form, enabling its implementation for computational processing. This is then used to implement the PID (proportional integral derivative) controller in a discrete time simulation, which adjusts the control signal $u_c(t)$ to track a step input r(t). The PID controller is implemented as follows:

Error calculation: The error e(t) is calculated and is the difference between the reference input r(t) and the previous system output y(t-1):

$$e(t) = r(t) - y(t-1)$$

Proportional Term (P): This term $u_p(t)$ is then calculated using the proportional gain K_p:

$$u_p(t) = K_p * e(t)$$

Integral Term (I): This term $u_i(t)$ is calculated by integrating the error over time, the error integral $e_i(t)$ is updated using the sampling time:

$$e_i(t) = e_i(t-1) + T_s * e(t)$$

Then the integral term is shown below, where K_i is the integral gain:

$$u_i(t) = K_i * e_i(t)$$

Derivative Term (D): The derivative term $u_d(t)$ is calculated by differentiating the error with respect to time:

$$e_d(t) = \frac{1}{T_s} * (e(t) - e(t-1))$$

Equation 3.1

Then the derivative term shown below, where K_d is the derivative gain:

$$u_d(t) = K_d * e_d(t)$$

The control signal $u_c(t)$ can then be calculated as the sum of the P, I and D terms:

$$u_c(t) = u_p(t) + u_i(t) + u_d(t)$$

Finally, the control signal is fed into the systems difference equation to compute the output y(t), calculated as:

$$y(t) = \frac{1}{a_0}(-a_1 y[t-1] - a_2 y[t-2] + b_0 u_c[t] + b_1 u_c[t-1] + b_2 u_c[t-2])$$

The PID controller closes the feedback loop by continuously adjusting the control input based on the error between the reference signal and the previous system output.

To asses and compare the performance of both the discrete and continuous control systems, standard performance indices are calculated which quantify how well the controller tracks the reference signal while also considering control effort and smoothness. The following performance metrics are used [3]:

Integral of Squared Error (ISE) - penalises large errors and emphasises overall tracking accuracy:

$$ISE = \sum e(t)^2 * T_s$$

Integral of the Time Weighted Absolute Error (ITAE) - emphasises errors which persist over time:

$$ITAE = \sum t * |e(t)| * T_s$$

Integral of the Absolute Control Effort (IACE) - measures how much control action is being applied:

$$IACE = \sum |u_c(t)| * T_s$$

Integral of the Absolute Control Effort Rate (IACRE) - shows how rapidly the control signal changes, which is relevant for actuator wear or instability:

$$IACER = \sum \left| \frac{u_c(t) - u_c(t-1)}{T_s} \right|$$

Multiplying the metrics by the sampling period T_S is essential to make sure the metrics are normalised across different sampling frequencies, without this scaling, comparisons between systems with different time steps would be skewed. With this setup the K_p, K_i and K_d values and sampling rates can be changed easily and both the discretised and continuous system's step response information, control signal, error metrics and root locus can be evaluated to design a suitable controller for the system.

3.2 Design of the PID controller

Different controller configurations were compared in this section which were found through iterative manual tuning, MATLABs PID tuning function and a method of refining after root locus pole placement.

Pole Placement (system 1):

Based on the design criteria the desired poles can be calculated [4,5]:

Using the percentage overshoot (M_p) as 5% and Settling time (t_s) as 1.8s both the natural frequency (ω_n) and damping (ζ) ratio are calculated using standard formulas:

$$\zeta = \frac{-\ln(M_p)}{\sqrt{\pi^2 + \left(ln(M_p)\right)^2}} = \frac{-\ln(0.05)}{\sqrt{\pi^2 + \left(ln(0.05)\right)^2}} \approx 0.69$$

$$t_s = \frac{4}{\zeta\omega_n} \rightarrow \omega_n = \frac{4}{\zeta t_s} = \frac{4}{0.69 * 1.8} \approx 3.22\ rad/s$$

The desired closed loop pole can then be found using the equation for poles of a second order system:

$$s = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2} = -2.22 \pm 2.33j$$

Given the initial system, from setting the numerator and denominator to zero and solving for 's', the system has poles s = -2.6980 +- 2.3603i and a zero at s = -119.35. The zero is in the far left plane and therefore has minimal effect on the dominant low frequency dynamics, therefore, the system is approximated as:

$$G_p(s) = \frac{4.774}{s^2 + 5.396s + 12.85}$$

A PID controller in the form:

$$C(s) = K * \frac{(s+a)^2}{s}$$

was selected therefore introducing a repeated zero at s = -a, along with a pole at the origin for integral action.

Using the root locus angle condition shown below, the angle contribution from the controller zeroes are found. The angles are be found from the perspective of s_d = -2.22 + 2.33j.

$$\sum \angle(zeroes) - \sum \angle(poles) = -180°$$

Angle from controller pole at zero:

$$\theta_1 = \angle(s_d - 0) = \angle(-2.22 + 2.33j) = \tan^{-1}\left(\frac{2.33}{-2.22}\right) \approx 133.6°$$

Angle from initial system poles p_1 = −2.698+2.3603j and p_2 = −2.698−2.3603j:

$$\theta_2 = \angle(s_d - p_1) = \angle(0.478 + 0.0303j) \approx -3.63°$$

$$\theta_3 = \angle(s_d - p_2) = \angle(-0.478 + 4.6903j) \approx 84.18°$$

Total angle from poles:

$$\angle_{poles} = 133.6° - 3.63° + 84.18° = 214.17°$$

Required angles from zeroes:

$$\angle_{zeroes} - 214.17° = -180° \rightarrow \angle_{zeroes} = 34.17°$$

With both zeroes located at the same point, each zero must contribute 17.08°, leading to:

$$a = 2.22 + \frac{2.33}{\tan(17.08°)} \approx 9.8$$

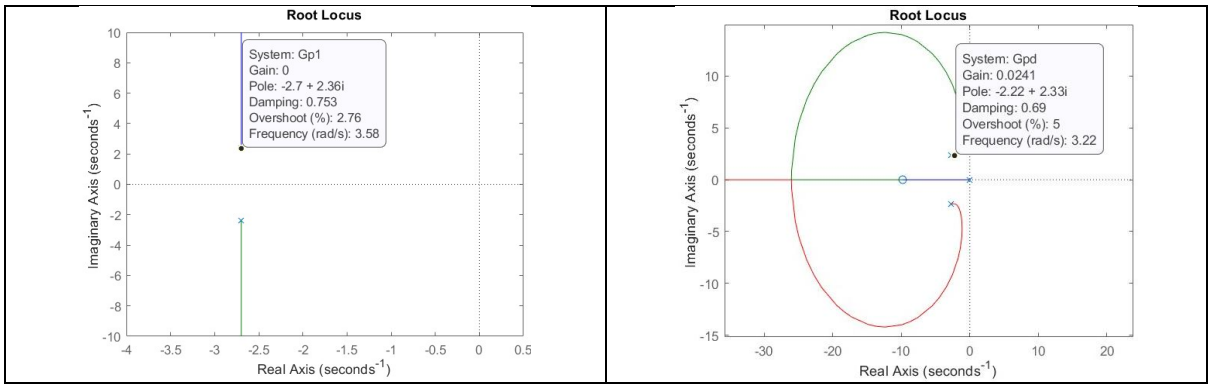
When looking at the initial system locus (figure 3.1 (a)) it is clear that the desired poles are not on the locus, even when the zero at -119 is included the locus curves to the left, however, after PID implementation (figure 3.1 (b)) these points are on the locus with the correct overshoot and damping.

From the locus in figure 3.1 (b) the gain is shown as 0.0241. Converting the controller to standard PID form:

$$C(s) = K\left(s + 2a + \frac{a^2}{s}\right) \rightarrow K_d = K,\ K_p = 2aK,\ K_i = a^2K$$

Substituting values gives controller gains:

$$K_d = 0.0242,\ \ K_p = 0.474,\ \ K_i = 2.32$$



(a) (b)

Figure 3.1 – Design stage Root Locus

These values were then refined based on table 3.2 to fit the design criteria. The step response information and error metrics are shown in table 3.3 for both the discrete and continuous time systems. Although the controllers are designed in continuous time for the continuous system, when discretised the system dynamics can change.

Par	Rise time (t_r)	Overshoot (M_p)	Steady-state error (e_{ss})	Settling time (t_s)	Stability
K_p	Decrease	Increase	Decrease	Small Change	Decrease
K_i	Decrease	Increase	Significant Decrease	Increase	Decrease
K_d	Minor Decrease	Decrease	Small Change	Decrease	Increase

Table 3.2 – Effects of Controller Gains

Time Domain	Rise time (s)	Settling time (s)	Maximum overshoot (%)	Steady-state error (%)	ISE	ITAE	IACE	IACRE	Meets criteria
Continuous	1.16	1.79	0.45	0	0.61	0.46	25.07	2.62	YES
Discrete	1.10	1.73	1.35	0	0.61	0.53	25.77	68.00	YES

Table 3.3 – System characteristics with PID controller implemented

Figure 3.2 shows the step response and control signal of the system in continuous and discrete time. It can be seen that both the discrete and continuous systems meet the design requirements for all metrics. Overall, there is minimal difference between the continuous and discretised system, the discrete system has a slightly faster rise time and settling time with a higher overshoot. This is due to the discrete derivative term reacting more sharply at the start of the response due to the difference approximation. The steady state error is also 0%, showing effective tracking of the reference input. ISE and ITAE are very close showing overall system performance is preserved after discretisation. Figure 3.2 also shows that both systems closely follow each other and closely match the reference step input.

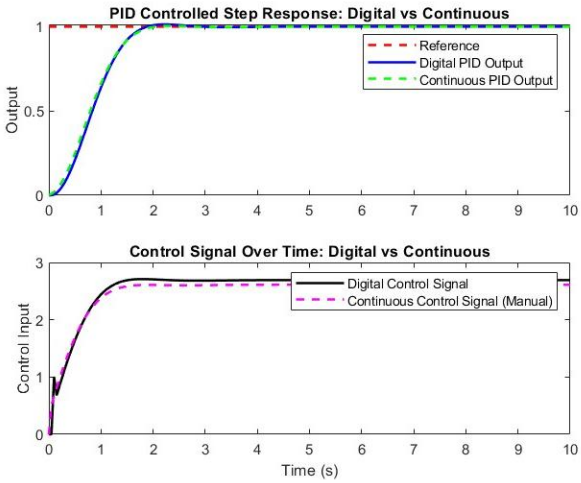


Figure 3.2 - Step response and controller signal for discrete and continuous system

A major difference is that the discrete system's IACRE value is far higher than the continuous. This represents the variability in the discrete control signal (shown in figure 3.2 with a small initial peak), and is common in all future simulations with the PID implemented and occurs because, as seen in the approximation of the derivative term (equation 3.1), any sudden change in the error (especially at the beginning where the step input happens instantly), a large derivative is caused because the difference is taken between two points. When sampling time is smaller this effect is also amplified due to dividing by a smaller number, the effect is also amplified when

derivative gain is increased. This analysis shows the discrete system accurately replicates the continuous time system behaviour and meets the design specification.

MATLAB PID Tuner (system 2):

MATLAB's PID tuner was also used to find initial controller parameters in which the gains were found as:

$$K_d = 0.461,\ \ K_p = 5.67,\ \ K_i = 13.6$$

The controller gains were fine-tuned slightly (within +-10%) to better meet the design criteria, system characteristics are shown below (table 3.4 and figure 3.3) :

Time Domain	Rise time (s)	Settling time (s)	Maximum overshoot (%)	Steady-state error (%)	ISE	ITAE	IACE	IACRE	Meets criteria
Continuous	0.42	0.59	4.77	0	0.15	0.05	23.98	4.22	YES
Discrete	0.28	1.46	0.86	0	0.16	0.08	27.42	678.13	YES

Table 3.4 - System characteristics with PID controller implemented

It can be seen that both the discrete and continuous systems meet the design criteria. In comparison to the previous controller this PID produces a much faster response with very low rise and settling times due to the aggressive controller gains. The steady state error remains at zero and the ISE and ITAE values are very low showing excellent tracking performance. The discrete system's IACRE value however, is far larger which is visible in the large peak in the control signal (figure 3.3), once again due to the derivative term reacting heavily with the higher derivative gain. This could be a problem if considering actuator wear, and if assuming the actuator isn't ideal and system dynamics become slower the aggressive tuning could also command faster changes than the actuator can actually respond to which could increase overshoot.

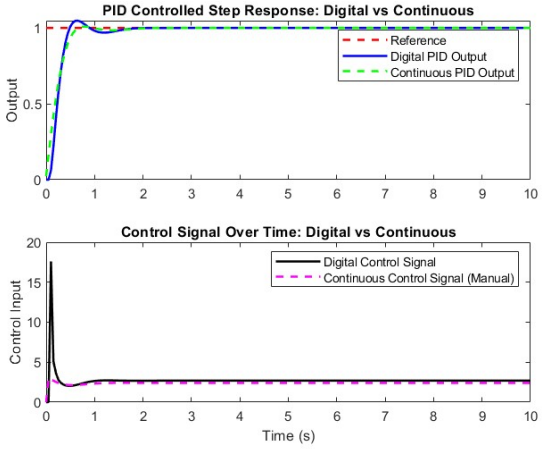


Figure 3.3 - Step response and controller signal for discrete and continuous system

Overall, the discrete system replicates the continuous behaviour closely, whilst meeting the design specifications, however, due to high max control effort (peak of control signal plot) the first system is preferred.

Iterative Method (system 3):

This method simply involved testing different PID values until the system step response met or exceeded the design criteria using table 3.2, initial values were chosen to be between the two previous controller deigns and depending on the step response the controller gains were altered accordingly. For example, if settling time was too long the integral term would be reduced or the derivative term would be increased. The error metrics and control signal were also taken into account in this process, for example high derivative coefficients would lead to high initial control action, therefore increasing system energy expenditure. Through this, the system step response characteristics were refined, and led to controller gain values of:

$$K_d = 0.24,\ \ K_p = 1.57,\ \ K_i = 4.9$$

The step response characteristics can be seen in table 3.5 and figure 3.4.

Time Domain	Rise time (s)	Settling time (s)	Maximum overshoot (%)	Steady-state error (%)	ISE	ITAE	IACE	IACRE	Meets criteria
Continuous	1.04	1.68	0	0	0.34	0.25	25.20	2.57	YES

Discrete	0.93	1.66	0	0	0.34	0.28	26.55	243.52	YES

Table 3.5 - System characteristics with PID controller implemented

It can be seen that both the discrete and continuous systems meet the design requirements. Similar trends are observed in this system as the previous systems, with the continuous and discrete systems being very similar in all metrics other than max control effort and IACRE. This system has lower settling and rise time than system one, whilst having less overshoot than both previous systems (0%). The ISE and ITAE values are also lower than that of system one and the IACRE and max control effort are significantly lower than system two's, suggesting less stress on the actuator and less energy expenditure.

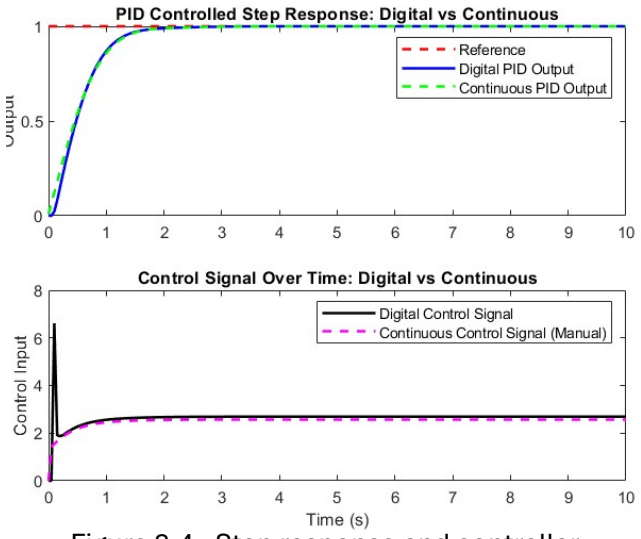


Figure 3.4 - Step response and controller signal for discrete and continuous system

Overall, this controller strikes the best balance between performance and control effort out of the three models tested. The rise and settling times are reasonable and exceed the design criteria, overshoot is eliminated, control effort is moderate. Therefore, this controller will be used for further implementation.

3.3 Stability analysis of the discrete and continuous system

This section compares the continuous time and discrete time root locus plots performed on the open loop system with the new PID controller implemented to assess the closed-loop pole locations as gain varies. Key differences in damping and frequency are analysed and system stability are also evaluated.

System Stability:

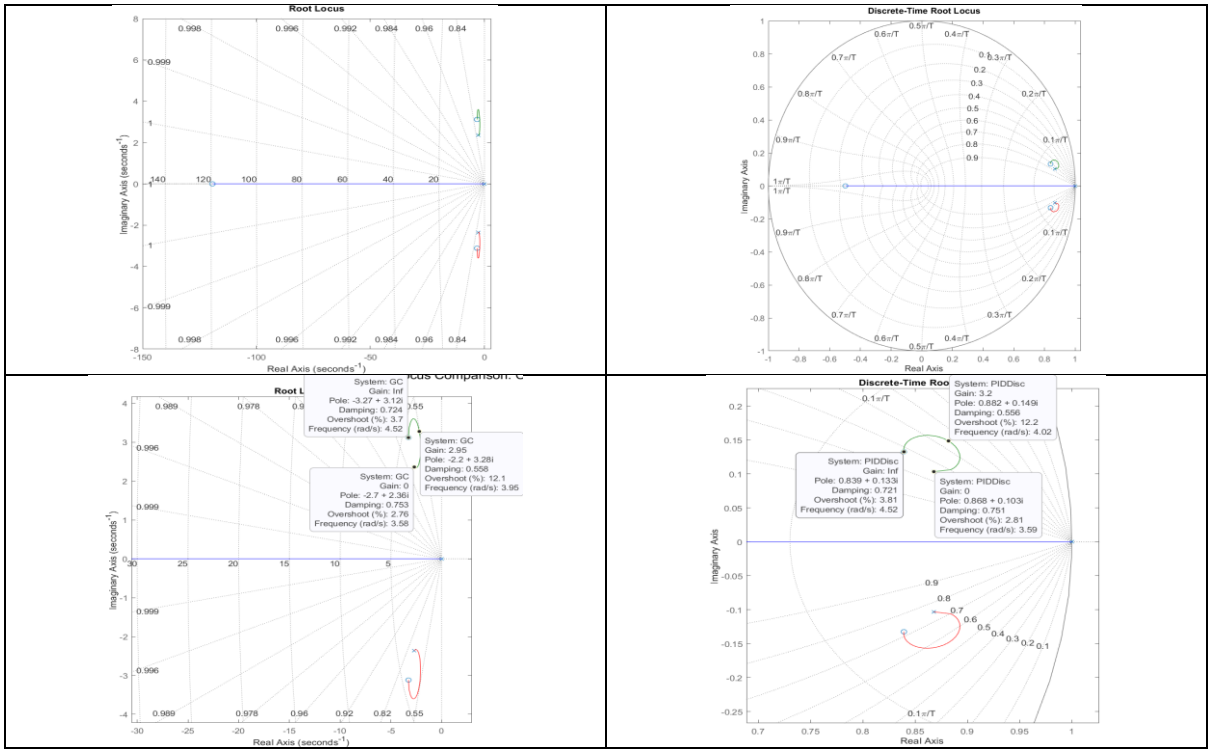
For the continuous time system (figure 3.5 (a)), stability is maintained for all positive gain values as all root locus branches lie entirely in the left half plane. The discrete time system (figure 3.5 (b)), stability is also preserved for all positive gain values as all root locus branches remain inside the unit circle. Therefore, the system with this PID controller is stable for all positive gains in both continuous and discrete domains. The gain (Gm) and phase margins (Pm) are also visible in the bode plots in figure 3.6, both the discrete and continuous systems show no difference in their Gm or Pm being infinite and 80.3deg respectively. Their shape does differ likely due to the continuous system more accurately capturing the less dominant high frequencies. These margins suggest that the system is highly stable in terms of open loop gain increase and shouldn't become unstable no matter how much the gain is increased, it also shows the system can tolerate a large amount of phase lag before instability.

Damping Characteristics:

The continuous and discrete time systems follow very similar damping characteristics, both follow a pattern of decreasing damping with increasing gain until a point, after which the damping increases as gain approaches infinity, this trend is depicted with values shown in figure 3.5. Only slight shifts in damping value and location can be observed which is likely due to slight frequency warping from the Tustin (bilinear) transformation used in the MATLAB code.

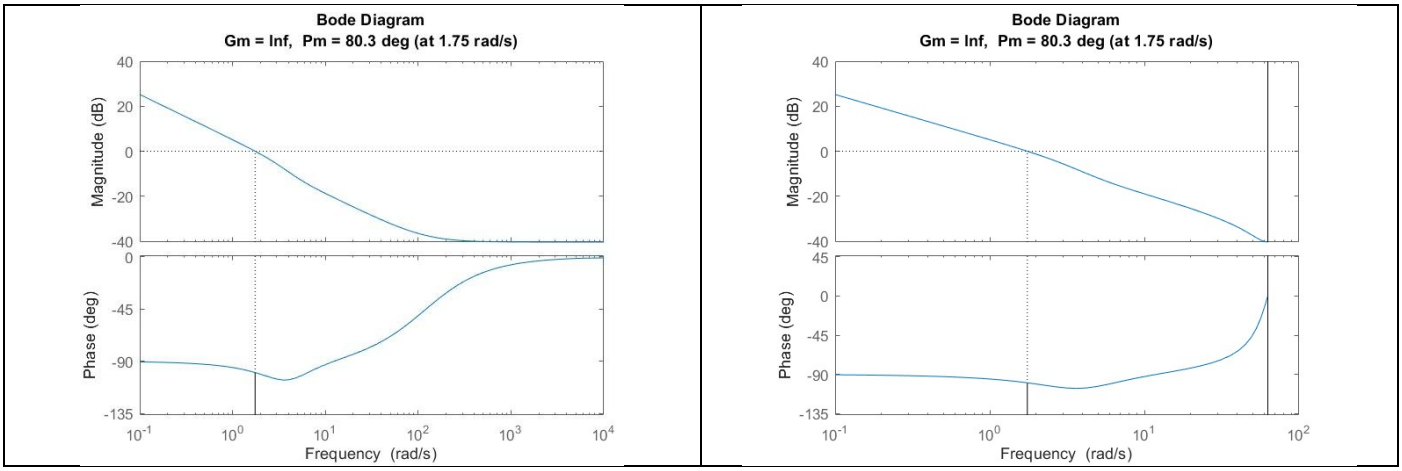
Frequency Characteristics:

The natural frequencies can also be seen in figure 3.5, it can be seen that once again the discrete and continuous systems are extremely similar across all gain values. The discrete frequencies show a slight upward shift initially however, both systems coverage to a natural frequency of 4.52 rad/s at infinite gain.



(a) – Continuous Time Locus (b) – Discrete Time Locus

Figure 3.5 – Root locus for continuous and discrete time systems



(a) Continuous Bode Plot (b) Discrete Bode Plot

Figure 3.6 – Bode plots for continuous and discrete time systems

Overall, this analysis demonstrates that both the discrete and continuous systems are stable and have very similar system dynamics, this further reinforces the suitability of this controller design.

3.3 Sampling Time Investigation

In digital control systems the sampling rate plays a critical role in system performance. A sufficiently fast sampling rate is required to accurately capture the dynamics of the continuous system. However, a sampling rate which is too fast could increase computational load with little benefit to system performance, therefore, an appropriate balance should be found.

The discretised PID implemented system was evaluated under different sampling times to assess its robustness to changes in sampling rate. The sampling periods was varied testing 1 ms, 20 ms, 50ms (initial system sampling time), 100 ms, 250 ms, and the root loci and step responses are shown in figure 3.7, system characteristics are also shown in table 3.6.

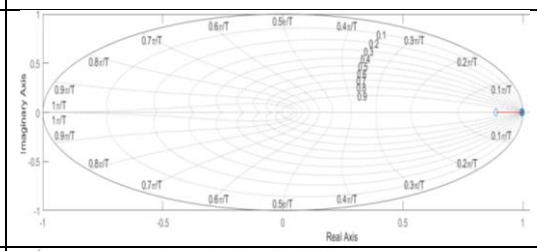
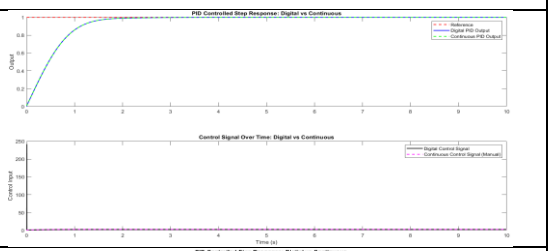
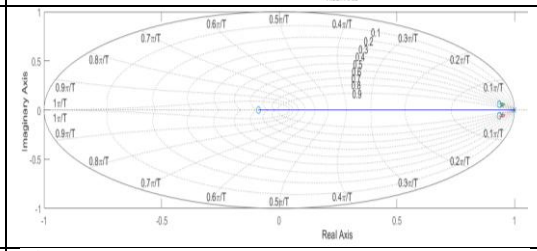
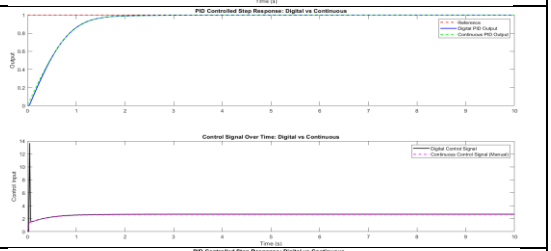
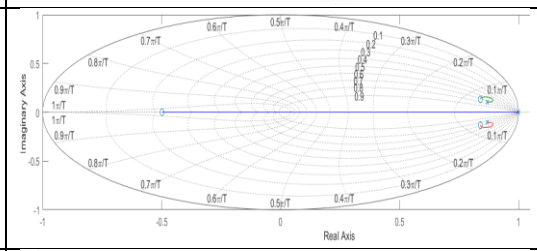
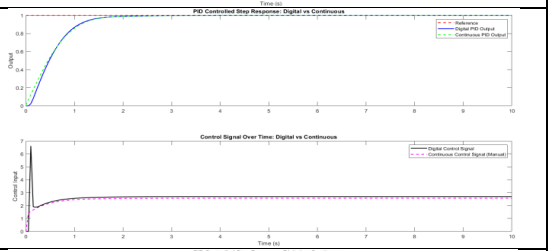
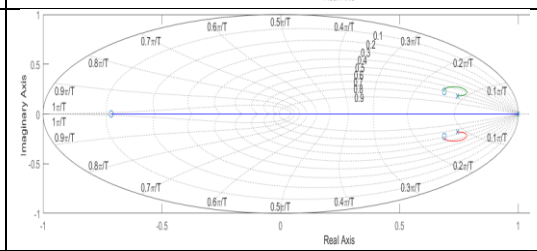
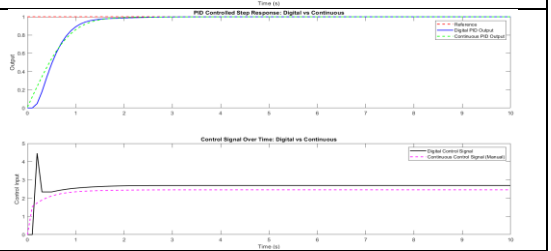
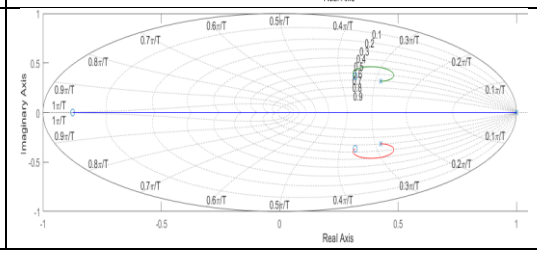
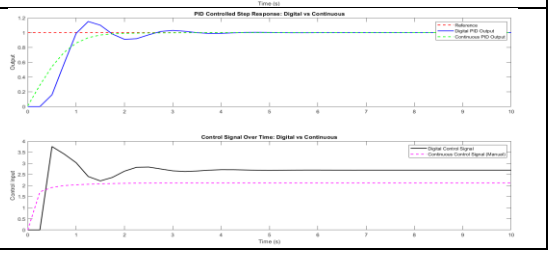
Sampling Period (ms)	Root Locus	Step Response
1		
20		
50		
100		
250		

Figure 3.7 – Root Loci and Step Response of system at different sample rates

The results in table 3.6 show that as the sampling time increases, the rise time decreases and the overshoot increases. This is because the courser sampling introduces a phase lag which effectively reduces damping since the system reacts to more outdated information and has to make large over corrections, making the system more oscillatory. This phenomenon is also visible in the step response in figure 3.7 where overshoot is very visible by 250ms sampling time. The design criteria was met for all sampling times tested other than 250ms which exceed both overshoot and settling time. When tested individually it was found that the system exceeds the settling time criteria at a sampling time of 130ms with 1.85 secs and exceeds overshoot criteria at 200ms with 5.38%.

Another large difference which can be seen, is in the performance indices where ISE and ITAE worsen as sampling slows which reflects the better tracking of the smaller sampling time, and is also seen in figure 3.7 where the discrete system follows the continuous better. The IACRE however, actually increases significantly with smaller sampling time, this is because as previously explained the derivative term is divided by the smaller time step therefore increasing control input. This is also reflected by the control signal shown in figure 3.7, which shows significant increase in max control effort as sampling time is reduced. This increase in IACRE could cause actuator wear or increased energy expenditure, however, in reality saturation to limit control input or a derivative filter would likely be used to counteract this.

The root loci in figure 3.7 show that the system stays stable even with high sampling times, another key point is that as sampling time increases the poles and zeros move to the left of the plot towards the origin of the z-plane. The happens because when the system is discretised the continuous poles are mapped using the relationship $z = e^{sTs}$, therefore a larger sampling period causes the poles to towards the zero [6]. This therefore, shows a loss of accuracy in tracking the continuous time system as opposed to the smaller sampling time systems which have their poles on the far right.

Sampling Time (ms)	Rise time (s)	Settling time (s)	Maximum overshoot (%)	Steady-state error (%)	ISE	ITAE	IACE	IACRE	Meets criteria
1	1.04	1.68	0	0	0.31	0.25	26.55	485,430	YES
20	1.00	1.67	0	0	0.32	0.26	26.55	1349.60	YES
50	0.93	1.66	0	0	0.34	0.28	26.55	243.52	YES
100	0.79	1.67	0	0	0.36	0.31	26.55	69.39	YES
250	0.53	3.21	15.05	0	0.48	0.72	26.55	25.01	NO

Table 3.6 – Effects of sampling time on system performance

Overall, a sampling time of 50-100ms is most viable as it balances computational cost and performance, this analysis also further reinforces the analysis done in section two where a sampling time of 50ms was chosen. The implemented controller is stable and exceeds the design criteria, keeps integral performance indices as low as possible whilst balancing computational cost with practical considerations such as actuator wear, making it a suitable choice to control the system.

4 Actuator Modelling and Saturation Effects

This section adds a more realistic actuator model into the system to assess its impact on closed loop behaviour. The response to increased setpoint amplitudes is simulated to highlight the effects of saturation on the system, the tolerable limits of performance degradation are also identified and possible design adaptations to mitigate the saturations effects are proposed and discussed.

The actuator is no longer considered ideal and its continuous time transfer function is described as:

$$G_{ac}(s) = \frac{1}{\tau s + 1}, \tau = 0.1, with \pm 20\ amplitude\ saturation$$

The actuator is cascaded with the system with the PID implemented before discretisation. The combined open loop system is then discretised together at the 50ms sampling time, because of this the system's discrete time transfer function becomes third order therefore needing a modified difference equation:

$$y[k] = \frac{1}{a_0}(-a_1 y[k-1] - a_2 y[k-2] - a_3 y[k-3] + b_0 u[k] + b_1 u[k-1] + b_2 u[k-2] + b_3 u[k-3])$$

This equation accounts for the additional actuator dynamics with the added a_3 and b_3 terms. To show the physical limitations of real actuators saturation is applied on the control input. To do this the PID generated control effort u_c is constrained between limits +- 20.

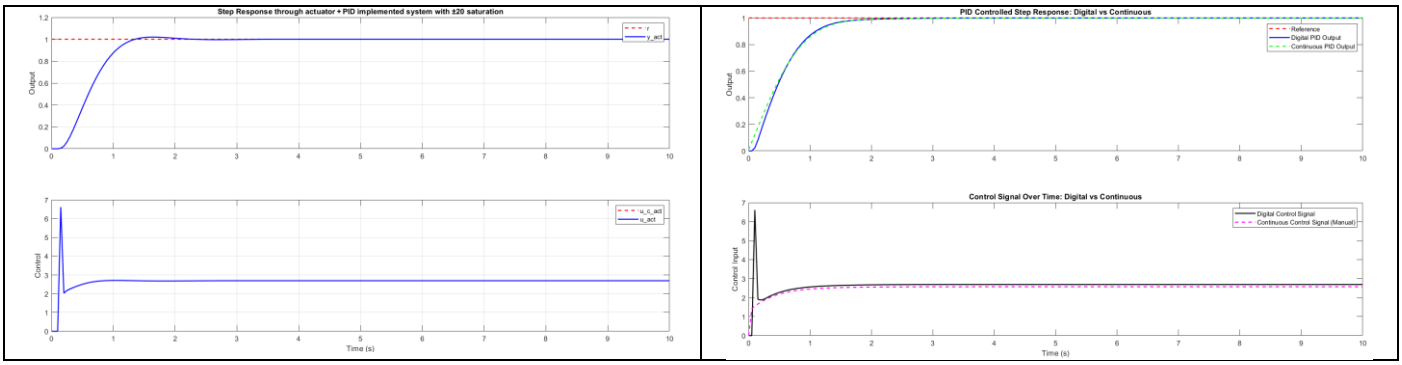
After application of the actuator and saturation the system performance changes and is quantified in table 4.1 and shown visually in figure 4.1 which compares each systems step response:

With actuator and saturation	Rise time (s)	Settling time (s)	Maximum overshoot (%)	Steady-state error (%)	ISE	ITAE	IACE	IACRE	Meets criteria
NO	0.93	1.66	0	0	0.34	0.28	26.55	243.52	YES
YES	0.76	1.25	2.00	0	0.40	0.29	26.68	238.47	YES

Table 4.1 – System performance characteristics with and without the actuator and saturation applied

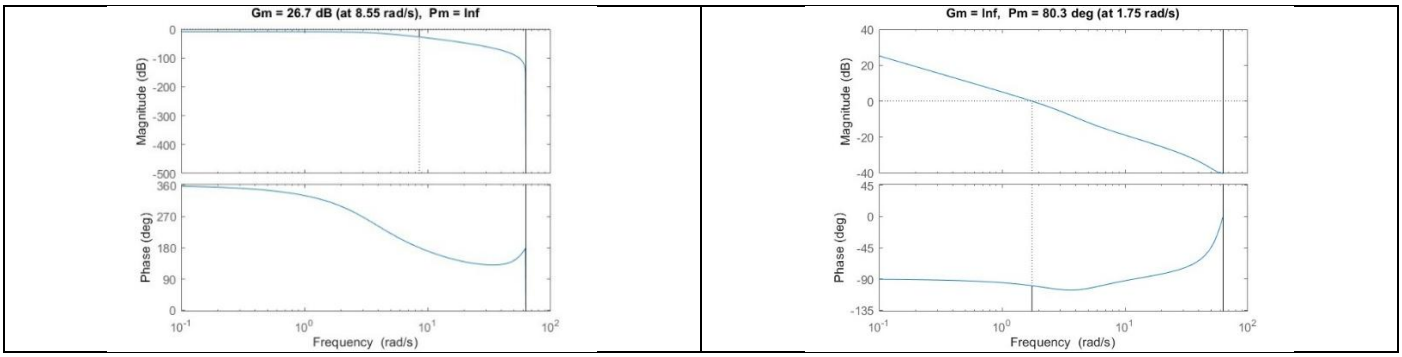
The actuator is modelled as a first order system which introduces a pole into the system at s = -10, when included the system becomes third order which adds a slight delay to the system therefore impacting the systems response and stability. The saturation of +- 20 was implemented, however, was not active as the control signal remained within these bounds throughout the simulation.

When the actuator was applied the settling time and rise time were reduced, and a small 2% overshoot was introduced, this can be seen in figure 4.1 (a), and can be attributed the actuators delay. For the same reason as seen in the sampling rate investigation the increased delay leads to overcorrections due to the controller reacting to outdated information.



(a) – System with actuator and saturation (b) – System with ideal actuator

Figure 4.1 – System step response with and without the actuator and saturation applied



(a) Bode plot with actuator (b) Bode plot with ideal actuator

Figure 4.2 – Bode plots with and without actuators

The bode plots for the system with the new actuator and ideal case are seen in figure 4.2. In the ideal system the 0 dB crossover happens when the phase is around -100 deg, therefore giving a phase margin of 80.3 deg. Since phase never reaches -180 deg while the gain is positive the gain margin is infinite. After introducing the actuator the additional pole causes the gain to decrease earlier and the phase to drop more rapidly due to the added lag. This therefore changes the systems characteristics, for example at 8.55 rad/s the phase crosses -180 deg, and the magnitude at that frequency corresponds to the gain margin of 26.7 dB, however, because the gain never crosses 0 dB the phase margin becomes infinite. The gain margin of 26.7 dB can be converted to linear scale and analysed using the following method [7]:

$$Gm\ (dB) = -20 \log_{10}(|L(j\omega)|)$$

Using this the magnitude at the point of instability is:

$$|L(j\omega)| = 10^{-\frac{26.7}{20}} \approx 0.046$$

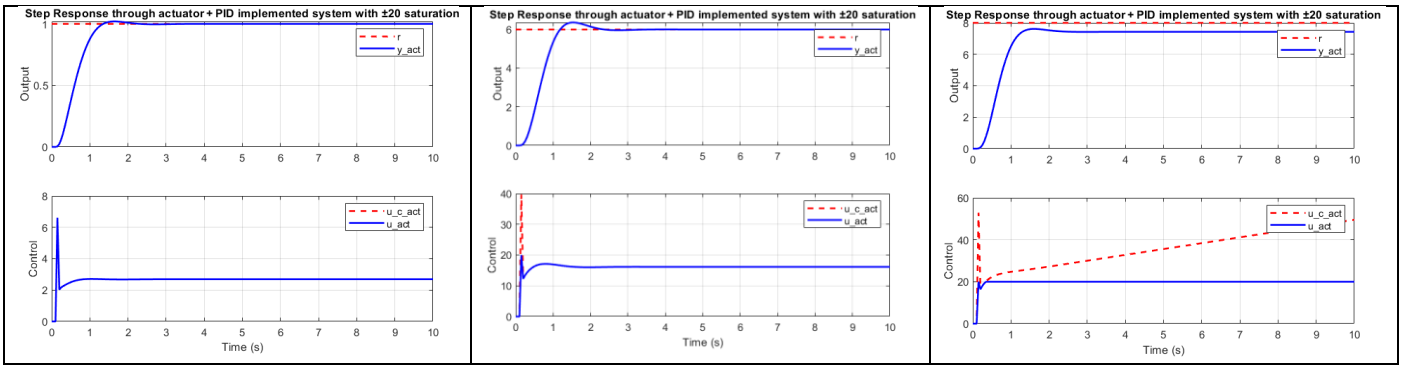
Therefore the systems gain could theoretically be increased by a factor of 21.7 (1/0.046) before becoming unstable. The results suggest that with the actuator the system becomes more robust against phase lag and can tolerate any phase lag before becoming unstable, however although the gain margin is high excessive gain increase could lead to instability.

4.1 Setpoint amplitude investigation

The setpoint amplitude is varied to determine system performance at different amplitudes. Table 4.2 shows that at low amplitudes (1-2) the system performs well maintaining the low overshoot, rise time, settling time and zero steady state error. However, as the setpoint amplitude increases (4-6) the initial spike in control action hits actuator saturation limits as seen in figure 4.3 (b) leading to increased overshoot, longer settling times and degraded performance indices. At the highest setpoint (8) the actuator saturation is too significant and limits the control signal not only at the initial spike, shown in figure 4.3 (c), therefore causing a steady state error of 7.5% and failure to settle as it never reaches the reference value, error metrics also spike showcasing the worse system tracking. Testing showed that the system could handle a setpoint amplitude of 3.2 before exceeding the 1.8s settling time criteria.

Setpoint amplitude	Rise time (s)	Settling time (s)	Maximum overshoot (%)	Steady-state error (%)	ISE	ITAE	IACE	IACRE	Meets criteria
1	0.76	1.25	2	0	0.40	0.29	26.68	238.47	YES
2	0.76	1.25	2	0	1.56	0.57	53.36	476.95	YES
4	0.72	1.96	3.92	0	6.73	1.29	106.72	705.02	NO
6	0.68	2.05	6.07	0	16.01	2.15	160.08	676.55	NO
8	0.84	NaN	0	7.5	32.73	30.52	197.68	544.49	NO

Table 4.2 – System performance at different setpoint amplitudes



(a) Setpoint amplitude = 1 (b) Setpoint amplitude = 6 (c) Setpoint amplitude = 8

Figure 4.3 – System step response at different setpoint amplitudes

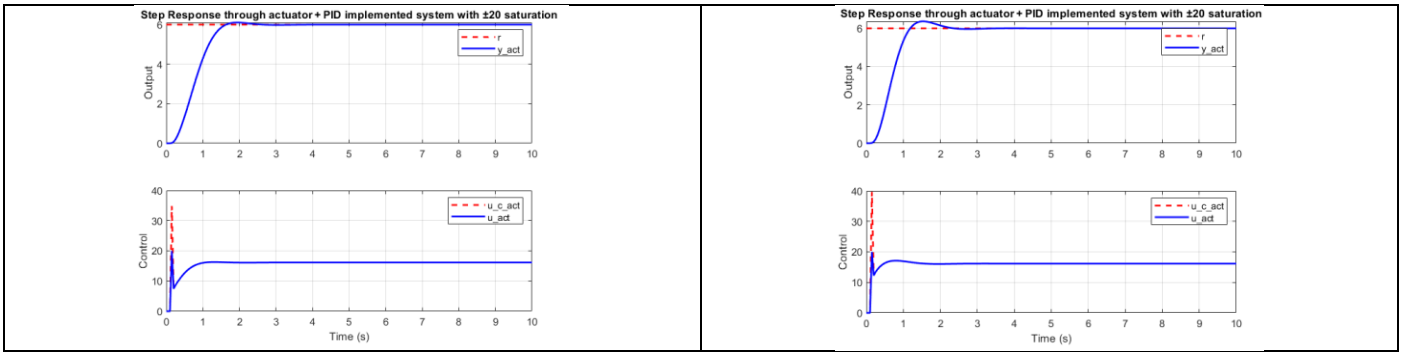
To mitigate the negative effects of increasing setpoint amplitude a solution should be employed. To do this two anti-wind up schemes were tested, the first of which was a simple clamping method limits the integral term of the PID controller to a maximum and minimum value, therefore preventing excessive integral windup by clipping the output of the integrator. The second was using a back calculation anti wind up method, in this, the difference between the saturated and unsaturated control effort is computed and fed back to correct the integrator's value, allowing the controller to unwind smoothly after saturation [8].

The results compiled in table 4.3 show that for small amplitudes (1-2) both methods did not affect the system response as the control signal remains below the limits of the saturation. At moderate amplitudes (4-6) the clamping method can actually give faster settling time and slightly lower overshoot and even improves upon the system characteristics at low amplitudes, however, to get these values the anti-wind up level manually was tuned for each amplitude, if one static value were to be used the clamped integrator would either wind up too much or clip too early. This makes the back calculation method more viable as it dynamically corrects for the error. Both methods fail at high amplitudes (8+) as no amount of anti-wind up can produce enough force to track the reference, hence the steady state error of 7.5% persists. Testing also showed that with the back calculation method implemented the system could tolerate amplitudes up to 7.5 before exceeding the design criteria whilst actually improving upon the system characteristics at low amplitudes.

Figure 4.4 also shows a comparison of the step responses with and without anti-wind up at setpoint amplitude of 6, in which it is clear that the control signal is less aggressive and the step response shows more desirable characteristics.

	Clamping Method				Back Calculation Method			
Setpoint Amplitude	Meets Criteria	Settling Time (s)	Overshoot (%)	Steady-state error (%)	Meets Criteria	Settling Time (s)	Overshoot (s)	Steady-state error (%)
1	YES	1.25	2.00	0	YES	1.25	2.00	0
2	YES	1.25	2.00	0	YES	1.25	2.00	0
4	YES	1.21	1.04	0	YES	1.41	1.61	0
6	YES	1.17	1.74	0	YES	1.52	1.79	0
8	NO	NaN	0	7.5	NO	NaN	0	7.5

Table 4.3 – Comparison of two different anti wind up schemes at different setpoint amplitudes



(a) Step response with anti-wind up (b) Step response without anti-wind up

Figure 4.4 – Comparison of step response with and without anti wind up at setpoint amplitude equal to 6

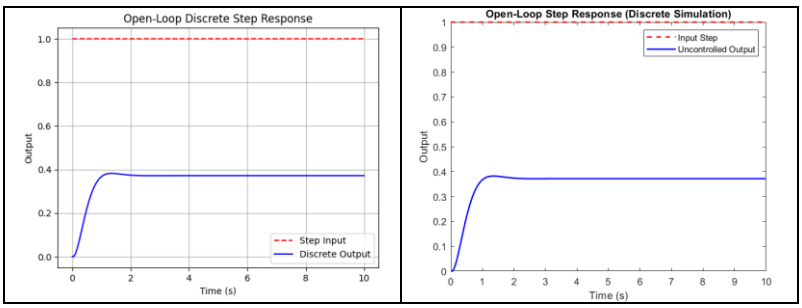
Overall, the system with the PID controller and the back calculation anti-wind up scheme is highly stable and surpasses the design criteria across all large range of setpoint amplitudes.

5 Python implementation

The system is implemented in python to eliminate MATLAB dependency. The same PID tuning, discretisation rules, anti-wind up logic are all contained within a single, reproducible python script. This allows for integration into a larger variety of systems which don't require expensive MATLAB licenses.

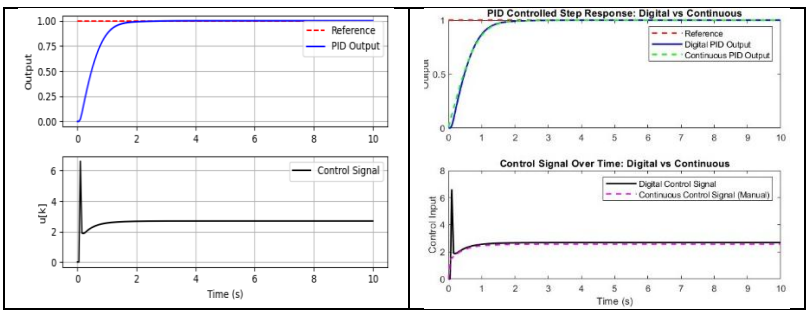
The python simulation follows the same implementation as the MATLAB code. The discretised open loop step response (figure 5.1) shows the fundamental system dynamics are followed by both languages with negligible differences.

The closed loop step response with the PID controller implemented (figure 5.2), shows that when the PID controller is implemented python accurately reproduces the MATLAB code's output. This is also shown in table 5.1



(a) Python (b) MATLAB

Figure 5.1 – Initial open loop discrete step responses

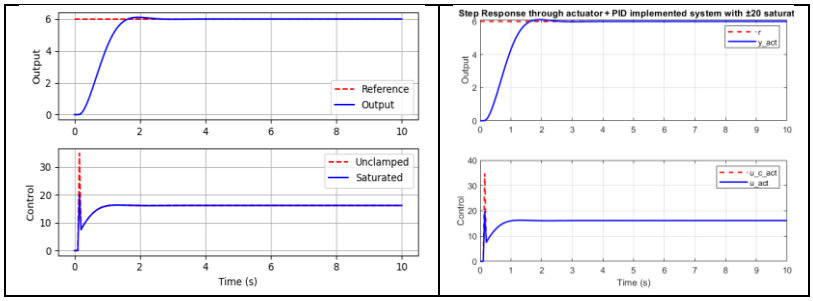


(a) Python (b) MATLAB

Figure 5.2 – Step response for discrete system with PID

which has very similar results, differing only slightly in rise and settling time.

Overall, python was able to replicate the MATLAB code with high accuracy, with the added benefits of open source flexibility and seamless integration into a variety of systems.



(a) Python (b) MATLAB

Figure 5.3 - Step response at 6 setpoint amplitude with PID, actuator and anti-wind up implemented

	Rise time (s)	Settling time (s)	Maximum overshoot (%)	Steady-state error (%)	ISE	ITAE	IACE	IACRE	Meets criteria
Python	0.90	1.70	0	0	0.34	0.28	26.55	243.52	YES
MATLAB	0.93	1.66	0	0	0.34	0.28	26.55	243.52	YES

Table 5.1 – Step response characteristics for discrete system with PID controller implemented

	Rise time (s)	Settling time (s)	Maximum overshoot (%)	Steady-state error (%)	ISE	ITAE	IACE	IACRE	Meets criteria
Python	0.90	1.55	1.79	0	18.93	2.63	157.43	833.44	YES
MATLAB	0.93	1.52	1.79	0	18.93	2.63	157.43	833.44	YES

Table 5.2 - Step response characteristics at 6 setpoint amplitude with PID, actuator and anti-wind up implemented

References:

1. Testbook. (2024). *Testbook.com - India's No.1 Govt Exam Preparation Site | Online Course | Mock Test*. [online] Available at: https://testbook.com/question-answer/in-a-control-system-the-bandwidth-of-a-system-rou--5f30107a80e56a0d095d3657 [Accessed 24 Apr. 2025].
2. Sarhadi, P. (2025), PID_Controller_Discrete_Time_Simulation.pdf [Lecture notes] School of Physics, Engineering and Computer Science, University of Hertfordshire, delivered via Canvas.
3. Idir, A., Zemmit, A., Akroum, H., Nesri, M., Guedida, S. and Canale, L. (2024). Enhancing Temperature Control of Electric Furnaces Using a Modified Pid Controller Design Strategy. [online] doi:https://doi.org/10.21203/rs.3.rs-4967918/v1.
4. Electrical4U (2021). Settling Time: What is it? (And How To Calculate it in MATLAB) | Electrical4U. [online] https://www.electrical4u.com/. Available at: https://www.electrical4u.com/settling-time/.
5. Mousavi, A. (n.d.). [online] Embedded Systems and Industrial Controller. Available at: https://www.brunel.ac.uk/~emstaam/material/bit/ESIC_5.pdf.
6. Fahri VATANSEVER and Metin HATUN (2021). s-to-z Transformation Tool for Discretization. Gazi Üniversitesi Fen Bilimleri Dergisi Part C Tasarım ve Teknoloji, [online] 9(4), pp.773–784. doi:https://doi.org/10.29109/gujsc.1003694.
7. Anon, (2022). Basics of Phase margin and Gain margin in Control Systems. [online] Available at: https://analogcircuitdesign.com/phase-and-gain-margin/.
8. Doroshenko, A. (2017). Problems of modelling Proportional–Integral–Derivative controller in automated control systems. MATEC Web of Conferences, 112, p.05013. doi:https://doi.org/10.1051/matecconf/201711205013.